

JPA Paso a Paso: Primer Ejemplo Funcional

¡Bienvenidos a un nuevo capítulo de nuestra serie "JPA Paso a Paso"! Hoy vamos a poner en práctica todo lo aprendido, conectando los conceptos de JPA con un ejemplo sencillo y funcional.

JAKARTA
PERSISTENCE AP
ORM com JPA

Preparando el Terreno: Un Vistazo Rápido

En videos anteriores, ya sentamos las bases cruciales para trabajar con JPA:



Base de Datos H2

Configuramos y entendimos la base de datos H2, nuestro motor ligero y eficiente para desarrollo.



Archivo persistence.xml

Exploramos el corazón de la configuración de JPA: el archivo `persistence.xml`.



Nuestras Entidades

Definimos nuestras clases de entidad, los objetos Java que mapearemos a tablas.

Ahora, es tiempo de ver cómo todo esto cobra vida en un flujo de trabajo completo.

Definiendo Nuestra Entidad: La Clase Persona

Para interactuar con la base de datos, primero necesitamos una representación Java. Aquí es donde entra nuestra entidad `Persona`, anotada correctamente para que JPA la reconozca.

```
@Entity
@Table(name="PERSONA")
public class Persona {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    @Column(name="NOMBRE", length=50, nullable=false)
    private String nombre;

    // Getters y Setters
}
```

Anotaciones Clave:

- `@Entity`: Marca la clase como una entidad JPA.
- `@Id`: Define la clave primaria de la entidad.
- `@GeneratedValue`: Configura la estrategia de generación de ID (aquí, auto-incremento).
- `@Column`: Personaliza el mapeo de campos a columnas de la tabla (nombre, longitud, si es nulo).

Iniciando JPA en el Código

Para comenzar a operar con la base de datos, necesitamos dos componentes fundamentales de JPA: el `EntityManagerFactory` y el `EntityManager`.



EntityManagerFactory

Se encarga de crear instancias de `EntityManager`.
Generalmente, se crea una sola vez por aplicación.

```
EntityManagerFactory emf =  
Persistence.createEntityManagerFactory("miUnidadPer  
sistencia");
```



EntityManager

Es la interfaz principal para interactuar con el contexto de persistencia. Se utiliza para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar).

```
EntityManager em = emf.createEntityManager();
```

Es vital cerrar tanto el `EntityManager` como el `EntityManagerFactory` cuando ya no se necesiten para liberar recursos.

Gestionando Cambios: Las Transacciones JPA

En JPA, todas las operaciones que modifican el estado de la base de datos (inserciones, actualizaciones, eliminaciones) deben realizarse dentro de una transacción. Esto asegura la atomicidad y la consistencia de los datos.

Flujo de una Transacción:

- `em.getTransaction().begin()`: Inicia la transacción. Marca el punto de partida de los cambios.
- `em.persist(objeto)`: Realiza la operación deseada (ej. persistir un objeto).
- `em.getTransaction().commit()`: Confirma todos los cambios realizados dentro de la transacción, haciéndolos permanentes en la base de datos.
- `em.getTransaction().rollback()`: En caso de error, se puede deshacer todos los cambios realizados desde el `begin()`.

`begin.` `commit`



"Las transacciones son la garantía de que tus datos permanecerán íntegros, incluso frente a fallos."

Persistiendo un Objeto: ¡Guardando Datos!

Una vez que tenemos nuestra entidad `Persona` y hemos iniciado una transacción, podemos guardar una instancia de `Persona` en la base de datos usando el método `persist()`.

```
// 1. Crear una instancia de la entidad
Persona nuevaPersona = new Persona();
nuevaPersona.setNombre("Ana López");

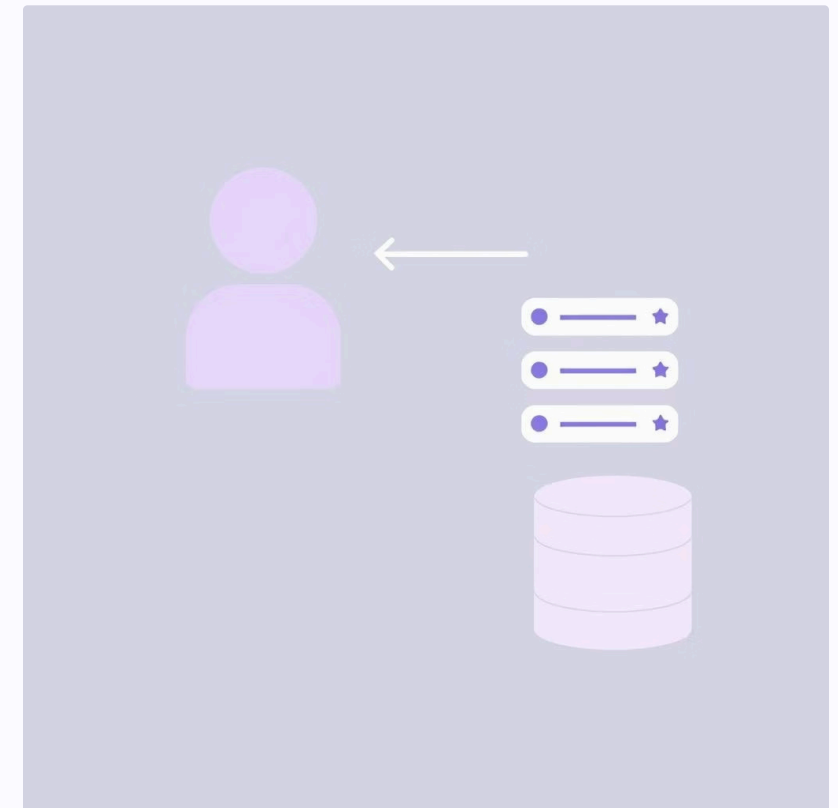
// 2. Iniciar la transacción
em.getTransaction().begin();

// 3. Persistir el objeto
em.persist(nuevaPersona); // ¡Aquí se guarda en la BD!

// 4. Confirmar la transacción
em.getTransaction().commit();

System.out.println("Persona guardada con ID: " +
nuevaPersona.getId());
```

El método `persist()` toma un objeto como argumento y lo añade al contexto de persistencia. Cuando la transacción se confirma, el estado de este objeto se sincroniza con la base de datos, traducándose en una sentencia `INSERT`.



Buscando un Objeto: Recuperando Datos

Para recuperar una entidad de la base de datos utilizando su clave primaria, empleamos el método `find()` del `EntityManager`. Este método es fundamental y eficiente.



```
// Suponemos que ya tenemos un ID de persona
Long idABuscar = 1L;

// Buscar el objeto por su ID
Persona personaEncontrada = em.find(Persona.class, idABuscar);

if (personaEncontrada != null) {
    System.out.println("Persona encontrada: " +
        personaEncontrada.getNombre());
} else {
    System.out.println("Persona no encontrada con ID: " + idABuscar);
}
```

Contexto de Persistencia:

Cuando utilizas `find()`, si el objeto con el ID solicitado ya está en el contexto de persistencia (es decir, ya ha sido cargado o persistido en la sesión actual), JPA lo retornará directamente sin ir a la base de datos. Si no está, lo recuperará de la base de datos y lo añadirá al contexto.

Observando el SQL: hibernate.show_sql

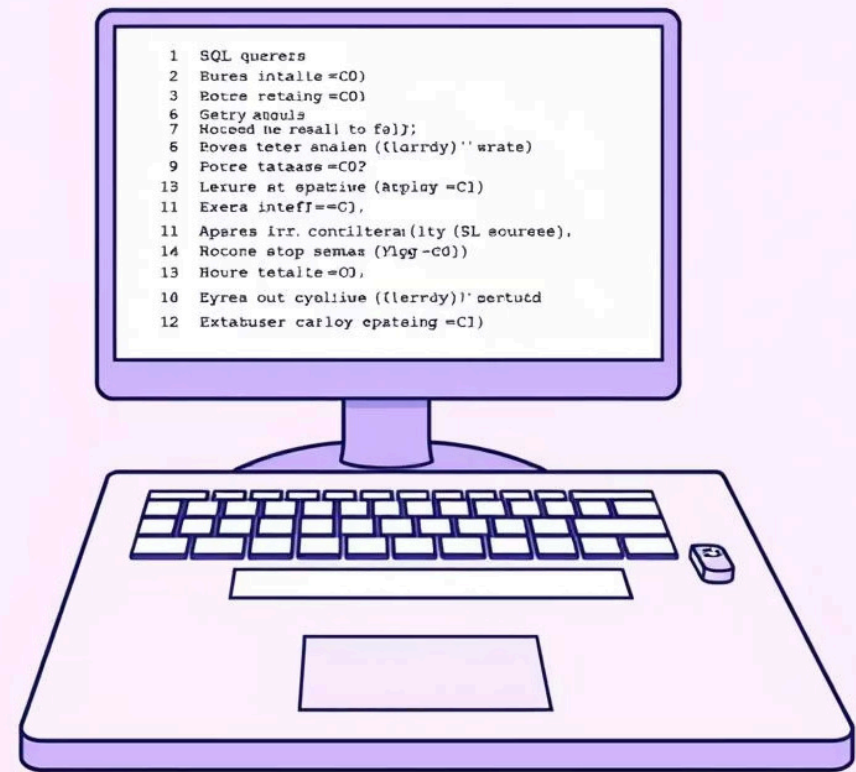
Una de las herramientas más útiles para depurar y entender cómo JPA interactúa con la base de datos es la propiedad `hibernate.show_sql`. Al activarla, verás las sentencias SQL que JPA genera en tu consola.

❗ ¿Cómo activarlo?

En tu archivo `persistence.xml`, dentro de las propiedades de la unidad de persistencia, añade:

```
<property name="hibernate.show_sql" value="true"/>
<property name="hibernate.format_sql" value="true"/>
```

Esto te permitirá ver claramente las operaciones `INSERT`, `SELECT`, `UPDATE` y `DELETE` que se ejecutan, ayudándote a validar que JPA está haciendo lo que esperas.

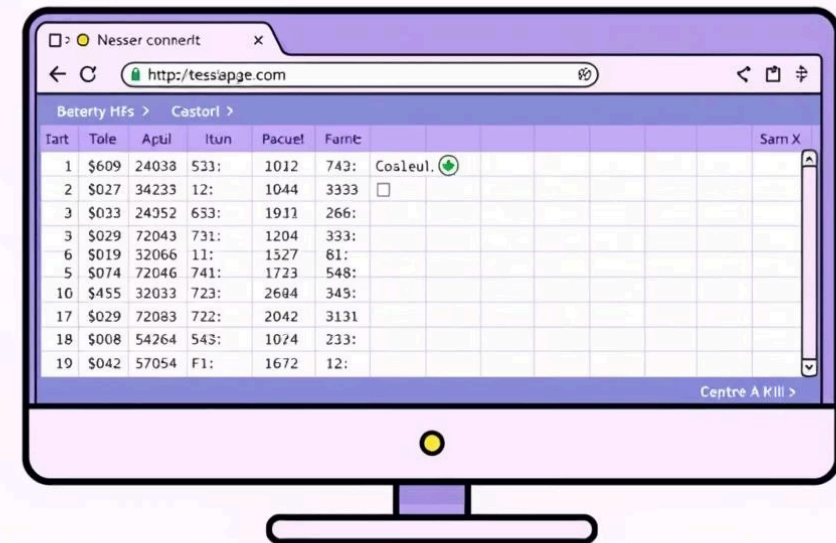


Verificando en la Consola H2

Para una confirmación visual de que los datos han sido guardados o modificados, la consola web de H2 es invaluable. Te permite navegar y consultar directamente las tablas de tu base de datos.

Pasos para Acceder:

1. Asegúrate de que tu aplicación JPA está en ejecución y la base de datos H2 está activa.
2. Abre tu navegador y ve a la URL de la consola H2 (usualmente `http://localhost:8082` o similar, según tu configuración).
3. Usa las credenciales y la URL JDBC de tu `persistence.xml` para conectarte.
4. Navega a la tabla `PERSONA` y ejecuta un `SELECT * FROM PERSONA;` para ver los registros.



The screenshot shows a web browser window with the URL `http://tessapge.com`. The page displays a table with 19 rows and 10 columns. The columns are labeled: Tart, Toile, Aptul, Itun, Pacuel, Farnic, Cosleul, and Sarn X. The first row has a green checkmark in the Cosleul column. The table is titled 'Beterty HFs' and 'Castorl'.

Tart	Toile	Aptul	Itun	Pacuel	Farnic	Cosleul	Sarn X
1	\$609	24038	533:	1012	743:	Cosleul. ☒	
2	\$027	34233	12:	1044	3333	☐	
3	\$033	24052	633:	1911	266:		
3	\$029	72043	731:	1204	333:		
6	\$019	32066	11:	1327	81:		
5	\$074	72046	741:	1723	548:		
10	\$455	32033	723:	2694	345:		
17	\$029	72083	722:	2042	3131		
18	\$008	54264	543:	1074	233:		
19	\$042	57054	F1:	1672	12:		

Esta visualización directa te da la certeza de que tus operaciones JPA se reflejan correctamente en la base de datos subyacente.

JPA en Acción: El Flujo Completo

Hemos recorrido el camino completo: desde la definición de una entidad hasta la interacción con la base de datos. Este es el ciclo básico de trabajo con JPA.



Próximos Pasos:

En nuestro siguiente video, profundizaremos en el universo de las anotaciones JPA (@Entity, @Table, @Id, @OneToMany, etc.), desglosando cada una y sus diversas configuraciones. ¡No te lo pierdas!